

REPORT CS 343 LAB 5

Group 38

- Raj Nandani Kumari: 230123048
- Neha Sellappan Devadas: 230123040
- Subhashree Kedia: 230123063
- Aditya Samal: 230123002

Our code files:  OS Lab Assignment 5 Code files

Task 1:

Extend the file system to allow much larger files by adding a doubly-indirect pointer.

Methodology and Code Overview

- ❖ Add a doubly-indirect pointer to the on-disk inode so an inode can address file blocks through: direct $\rightarrow$ single-indirect $\rightarrow$ doubly-indirect $\rightarrow$ data.
- ❖ Implement three-region block mapping (direct / single indirect / doubly indirect), allocate metadata blocks on demand, and recursively free all levels on truncate.
- ❖ Keep dinode on-disk size unchanged (128 bytes) by reducing NDIRECT and adding two pointer slots (single + double).

Where changes were made (files) and what was changed

fs.h

- ❖ Reduced **NDIRECT** (12 $\rightarrow$ 11).
 - Because the on-disk dinode is fixed. To add those pointers without increasing the on-disk inode size, we reduce the count of direct pointers (NDIRECT)
- ❖ Changed **dinode addrs[]** length to **NDIRECT + 2**
- ❖ Defined **NINDIRECT = BSIZE / sizeof(uint)**.
- ❖ Recomputed MAXFILE macro: NDIRECT + NINDIRECT + NINDIRECT^2 (blocks).
- ❖ Ensured **sizeof(struct dinode) == 128**.

fs.c

bmap(struct inode *ip, uint bn)

Returns the **disk block number** for the logical block index **bn**.

If the block doesn't exist, it allocates it (when allowed).

1. Direct Mapping ($0 \leq bn < NDIRECT = 11$)

- The inode's first 11 entries (**ip->addrs[0..10]**) directly store block numbers.
- If the block is missing (**0**), allocate a new one with **balloc()**.

2. Single-Indirect Mapping ($NDIRECT \leq bn < NDIRECT + NINDIRECT$)

- **ip->addrs[11]** stores the disk block number of the *single-indirect block*.
- That block holds an array of **NINDIRECT** (128) data-block addresses.

Steps:

1. If the indirect block doesn't exist, allocate it and zero it out.
2. Read it into the buffer cache with `bread()`.
3. Treat `bp->data` as a `uint*` array of block numbers.
4. Use index `bn - NDIRECT` to find or allocate the data block.
5. If new data block allocated, write changes (`log_write(bp)`) and release (`brelse()`).

3. Doubly-Indirect Mapping

- Applies when `bn` exceeds the single-indirect range.
- **Structure:**
`ip->addrs[12] --> [ indirect1, indirect2, ... ]` ← each points to an indirect block
`indirectX --> [ data1, data2, ... ]` ← each entry points to data blocks

Steps:

1. Compute position within doubly-indirect space:
`bn' = bn - NDIRECT - NINDIRECT;`
`dib_index = bn' / NINDIRECT;`
`dib_offset = bn' % NINDIRECT;`
2. If `ip->addrs[12]` (the doubly-indirect block) doesn't exist, allocate it and zero it.
3. Read the doubly-indirect block; treat as an array of indirect block pointers.
4. Fetch (or allocate) the corresponding indirect block using `dib_index`.
5. Read that indirect block; allocate the data block if `a[dib_offset] == 0`.
6. Write all modified blocks (`log_write()`), then `brelse()` to release buffers.

```
if (bn < NINDIRECT * NDINDIRECT) {
    uint dib_index = bn / NDINDIRECT;
    uint dib_offset = bn % NDINDIRECT;

    // Load/allocate doubly-indirect block
    if ((addr = ip->addrs[NDIRECT+1]) == 0) {
        addr = balloc(ip->dev);
        if (addr == 0)
            return 0;
        ip->addrs[NDIRECT+1] = addr;
    }

    // Read doubly-indirect block
    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;

    // Get the indirect block address from doubly-indirect block
    uint ib_addr = a[dib_index];
    if (ib_addr == 0) {
        ib_addr = balloc(ip->dev);
        if (ib_addr == 0) {
            brelse(bp);
            return 0;
        }
        a[dib_index] = ib_addr;
        log_write(bp);
    }
}
```

```

    brelse(bp);

    // Read the indirect block
    bp = bread(ip->dev, ib_addr);
    a = (uint*)bp->data;

    // Get the data block address from indirect block
    if((addr = a[dib_offset]) == 0){
        addr = balloc(ip->dev);
        if(addr){
            a[dib_offset] = addr;
            log_write(bp);
        }
    }
    brelse(bp);
    return addr;
}

panic("bmap: out of range");
}

```

Theoretical Capacity

- **NDIRECT = 11**
- **NINDIRECT = BSIZE / sizeof(uint) = 512 / 4 = 128**

Then total addressable blocks:

= NDIRECT + NINDIRECT + (NINDIRECT * NINDIRECT)

= 11 + 128 + 16384

= 16,523 blocks

At 512 bytes per block. Max file size $\approx 16,523 \times 512 = 8,455,296$ bytes ≈ 8.06 MB

itrunc(struct inode *ip) frees **all data blocks** associated with the inode when a file is deleted or truncated.

Demo/Test Code: [Click Here](#)

Output:

```

$ bigfile
bigfile: testing large file support (doubly-indirect blocks)
Creating a file with size 4194304 bytes...
bigfile: wrote 262144 bytes...
bigfile: wrote 524288 bytes...
bigfile: wrote 786432 bytes...
bigfile: wrote 1048576 bytes...
bigfile: wrote 1310720 bytes...
bigfile: wrote 1572864 bytes...
bigfile: wrote 1835008 bytes...
bigfile: wrote 2097152 bytes...
bigfile: wrote 2359296 bytes...
bigfile: wrote 2621440 bytes...
bigfile: wrote 2883584 bytes...
bigfile: wrote 3145728 bytes...
bigfile: wrote 3407872 bytes...
bigfile: wrote 3670016 bytes...
bigfile: wrote 3932160 bytes...
bigfile: wrote 4194304 bytes...

```

```

$ bigfile
bigfile: testing large file support (doubly-indirect blocks)
Creating a file with size 4194304 bytes...
bigfile: wrote 262144 bytes...
bigfile: wrote 524288 bytes...
bigfile: wrote 786432 bytes...
bigfile: wrote 1048576 bytes...
bigfile: wrote 1310720 bytes...
bigfile: wrote 1572864 bytes...
bigfile: wrote 1835008 bytes...
bigfile: wrote 2097152 bytes...
bigfile: wrote 2359296 bytes...
bigfile: wrote 2621440 bytes...
bigfile: wrote 2883584 bytes...
bigfile: wrote 3145728 bytes...
bigfile: wrote 3407872 bytes...
bigfile: wrote 3670016 bytes...
bigfile: wrote 3932160 bytes...
bigfile: wrote 4194304 bytes...

```

Task 2:

Task Overview

The goal of this task is to extend the xv6 file system to support symbolic (soft) links. A new system call `symlink(target, path)` will create a symbolic link at `path` pointing to `target`. File operations should transparently resolve symbolic links to access the target file or directory. This requires updating the inode structure and file system logic while maintaining compatibility with existing files.

Methodology and Code Overview

1. Adding Symbolic Link Support

- **New File Type**

In `kernel/stat.h`, a new file type constant was introduced:

```
#define T_SYMLINK 4 // New symbolic link type
```

This allows the inode structure to recognize and handle symbolic links distinctly from regular files, directories, and devices.

- **New Open Flag**

In `kernel/fcntl.h`, a new flag was defined:

```
#define O_NOFOLLOW 0x800
```

When `open()` is called with this flag, the kernel opens the symbolic link itself instead of following it to the target file.

2. Adding System Call Entry

In `kernel/syscall.h`, a new system call number was defined:

```
#define SYS_close 21
#define SYS_symlink 22
```

In `kernel/syscall.c`, the syscall was mapped and externally declared, linking the user-level `symlink()` function to the kernel implementation `sys_symlink()`.

3. Implementing `sys_symlink()`

The symbolic link creation logic is implemented in `kernel/sysfile.c`:

```
uint64
sys_symlink(void)
{
```

```

char target[MAXPATH], path[MAXPATH];
struct inode *ip;
// Get arguments: target path and link path
if(argstr(0, target, MAXPATH) < 0 || argstr(1, path, MAXPATH) < 0)
    return -1;
begin_op();
// Create a new inode of type T_SYMLINK
ip = create(path, T_SYMLINK, 0, 0);
if(ip == 0){
    end_op();
    return -1;
}

// Write the target path into the symlink inode's data
if(writei(ip, 0, (uint64)target, 0, strlen(target)) != strlen(target)){
    iunlockput(ip);
    end_op();
    return -1;
}
iunlockput(ip);
end_op();
return 0;
}

```

- Retrieves the target and link path arguments from user space.
- Allocates a new inode of type **T_SYMLINK**.
- Writes the target path into the data block of the symbolic link inode.
- Finalizes the operation by unlocking and ending the file system transaction.

4. Handling Symbolic Link Resolution in **sys_open()**

To ensure symbolic links behave transparently, modifications were made in **sys_open()** within **sysfile.c**.

When a symbolic link is encountered, the system recursively resolves it up to a maximum depth of 10 to prevent infinite loops:

```

if(ip->type == T_SYMLINK && !(omode & O_NOFOLLOW)){
    int depth = 0;
    while(ip->type == T_SYMLINK && depth < 10){
        char target[MAXPATH];
        int len;
        // Read the target path from the symlink
        if((len = readi(ip, 0, (uint64)target, 0, MAXPATH)) <= 0){
            iunlockput(ip);
            end_op();
            return -1;
        }
        // Null-terminate the target path
        if(len >= MAXPATH)
            len = MAXPATH - 1;
        target[len] = 0;
        iunlockput(ip);
    }
}

```

```

        // Follow the symlink
        if((ip = namei(target)) == 0){
            end_op();
            return -1;
        }
        ilock(ip);
        depth++;
    }
    // Check if we hit the recursion limit
    if(depth >= 10 && ip->type == T_SYMLINK){
        iunlockput(ip);
        end_op();
        return -1; // Too many levels of symbolic links
    }
}

```

- Ensures symlink resolution is transparent to users.
- Uses recursion depth control to avoid circular references.
- Honors the `O_NOFOLLOW` flag when provided.

5. Updating the `create()` Function

The `create()` function was updated to allow reusing existing symbolic link inodes when creating regular files:

```

if((ip = dirlookup(dp, name, 0)) != 0){
    iunlockput(dp);
    ilock(ip);
    if(type == T_FILE && (ip->type == T_FILE || ip->type == T_DEVICE || ip->type
        == T_SYMLINK))
        return ip;
    iunlockput(ip);
    return 0;
}

```

Allows symbolic links to coexist with regular files and devices during file creation without causing conflicts.

6. User-Space Integration

To make the new system call accessible from user programs:

In `user/user.h`:

```
int symlink(const char*, const char*);
```

In `user/usys.pl`:

```
entry("symlink");
```

This allows programs to invoke the new system call as:

```
symlink("target.txt", "link_to_target");
```

Demo/Test Code:

The goal of the demo code is to verify that: A symlink can be created successfully and reading a file via its symlink returns the correct data.

Test 1: Basic Symbolic Link Creation and Reading

- Create a normal test `testfile` and write "Hello, symlink!" to it.
- Create a symlink `testlink` pointing to `testfile`.
- Open `testlink` and read data.
- Verify that the content matches the original file.
- Delete both the link and the file.

```
fd = open("testfile", O_CREATE | O_WRONLY);
if(fd < 0){
    printf("FAILED: cannot create testfile\n");
    return;
}
write(fd, "Hello, symlink!", 15);
close(fd);
```

Test 2: Writing Through a Symbolic Link

- Create empty file `writefile`.
- Create `writelink` → `writefile`.
- Open `writelink` and write "Written via symlink".
- Open the original file `writefile` and verify it contains the same string.
- Clean up.

```
fd = open("writefile", O_CREATE | O_WRONLY);
if(fd < 0){
    printf("FAILED: cannot create writefile\n");
    return;
}
close(fd);
printf("[Step 1] Created empty 'writefile'.\n");
```

```
fd = open("writelink", O_WRONLY);
if(fd < 0){
    printf("FAILED: cannot open writelink for writing\n");
    unlink("writefile");

    unlink("writelink");

return; }
```

Test 3: Circular Symbolic Link Detection

- Create `link2` → `link1`.
- Create `link1` → `link2`. (Now they point to each other — a loop!)
- Try opening `link1`. It should **fail** (prevent infinite recursion).
- Clean up both links.

```
if(symlink("link1", "link2") < 0){
    printf("FAILED: cannot create link2\n");
    return;
}
printf("[Step 1] Created link2 -> link1.\n");

if(symlink("link2", "link1") < 0){
    printf("FAILED: cannot create link1\n");
    unlink("link2");
    return;
}
printf("[Step 2] Created link1 -> link2 (circular reference).\n");

fd = open("link1", O_RDONLY);

if(fd >= 0){
    printf("FAILED: opened circular symlink (should fail)\n");
    close(fd);
    unlink("link1");
    unlink("link2");
    return;
}
```

Test 4: `O_NOFOLLOW` Flag Test

- Create a target file `targetfile`.
- Create a symlink `nofollowlink` → `targetfile`.
- Open the symlink with `O_RDONLY | O_NOFOLLOW`.
- Use `fstat()` to verify the opened file's type is `T_SYMLINK` (not regular).
- Clean up.

```
fd = open("targetfile", O_CREATE | O_WRONLY);
if(fd < 0){
    printf("FAILED: cannot create targetfile\n");
    return;
}
write(fd, "target content", 14);
close(fd);
printf("[Step 1] Created target file.\n");

if(symlink("targetfile", "nofollowlink") < 0){
    printf("FAILED: cannot create nofollowlink\n");
    unlink("targetfile");
    return;
}

printf("[Step 2] Created symlink 'nofollowlink' -> 'targetfile'.\n");
```



```

fd = open("nofollowlink", O_RDONLY | O_NOFOLLOW);
if(fd < 0){
    printf("FAILED: cannot open symlink with O_NOFOLLOW\n");
    unlink("targetfile");
    unlink("nofollowlink");
    return;
}
printf("[Step 3] Opened symlink with O_NOFOLLOW flag.\n");

if(fstat(fd, &st) < 0){
    printf("FAILED: cannot stat symlink\n");
    close(fd);
    unlink("targetfile");
    unlink("nofollowlink");
    return;
}

close(fd);

if(st.type != T_SYMLINK){
    printf("FAILED: O_NOFOLLOW did not open symlink itself (type=%d)\n", st.type);
    unlink("targetfile");
    unlink("nofollowlink");
    return;
}

```

Test 5: Unlinking a Symbolic Link

- Create target **unlinkfile**.
- Create **unlinklink** → **unlinkfile**.
- Unlink (delete) **unlinklink**.
- Verify **unlinkfile** still exists and is accessible.
- Clean up.

```

fd = open("unlinkfile", O_CREATE | O_WRONLY);
if(fd < 0){
    printf("FAILED: cannot create unlinkfile\n");
    return;
}
write(fd, "data", 4);
close(fd);
printf("[Step 1] Created target file 'unlinkfile'.\n");

if(symlink("unlinkfile", "unlinklink") < 0){
    printf("FAILED: cannot create unlinklink\n");
    unlink("unlinkfile");
    return;
}
printf("[Step 2] Created 'unlinklink' -> 'unlinkfile'.\n");

if(unlink("unlinklink") < 0){
    printf("FAILED: cannot unlink symlink\n");
}

```

```
    unlink("unlinkfile");  
    unlink("unlinklink");  
  
    return;  
}
```

Test 6: Chained Symbolic Links

- Create a file `finalfile` with content "`final data`".
- Create `link3` → `finalfile`.
- Create `link2` → `link3`.
- Create `link1` → `link2`.
- Open `link1` and read data.
- Verify the correct content is read.
- Clean up all links and files.

Output:

```

=====
Starting Symbolic Link Tests in xv6
=====

=====
TEST 1: Basic symbolic link creation and reading
=====
[Step 1] Created file 'testfile' with content.
[Step 2] Created symbolic link 'testlink' -> 'testfile'.
[Step 3] Opened 'testlink' for reading.
[Step 4] Read content through symlink successfully.
[Step 5] Cleaned up 'testfile' and 'testlink'.
PASSED: Basic symbolic link test

=====
TEST 2: Writing through symbolic link
=====
[Step 1] Created empty 'writefile'.
[Step 2] Created 'writelink' -> 'writefile'.
[Step 3] Wrote content through symlink.
[Step 4] Verified content written in target file.
[Step 5] Cleaned up 'writefile' and 'writelink'.
PASSED: Write through symbolic link test

=====
TEST 3: Circular symbolic link detection
=====
[Step 1] Created link2 -> link1.
[Step 2] Created link1 -> link2 (circular reference).
[Step 3] Correctly failed to open circular symlink.
[Step 4] Cleaned up circular links.
PASSED: Circular symlink detection test

=====
TEST 4: O_NOFOLLOW flag test
=====
[Step 1] Created target file.
[Step 2] Created symlink 'nofollowlink' -> 'targetfile'.
[Step 3] Opened symlink with O_NOFOLLOW flag.
[Step 4] Cleaned up test files.
PASSED: O_NOFOLLOW flag test

=====
TEST 5: Unlinking symbolic link
=====
[Step 1] Created target file 'unlinkfile'.
[Step 2] Created 'unlinklink' -> 'unlinkfile'.
[Step 3] Unlinked the symlink successfully.
[Step 4] Verified target file still exists.
[Step 5] Cleaned up target file.
PASSED: Unlink symbolic link test

=====
TEST 6: Chained symbolic links
=====
[Step 1] Created 'finalfile' with content.
[Step 2] Created link3 -> finalfile.
[Step 3] Created link2 -> link3.
[Step 4] Created link1 -> link2.
[Step 5] Opened chained symlink (link1).
[Step 6] Successfully read through chain.
[Step 7] Cleaned up all chained files.
PASSED: Chained symbolic links test

```

The output shows the results of running the `symlinktest` program, which verifies all major functionalities of symbolic links in xv6.

Test 1 – Basic Creation and Reading:

A symbolic link `testlink` → `testfile` was created and successfully read through the link.

Test 2 – Writing Through Link:

Data written to a link (`writelink`) correctly appeared in the target file.

Test 3 – Circular Link Detection:

A loop (`link1` ↔ `link2`) was detected, and opening was safely prevented.

Test 4 – O_NOFOLLOW Flag:

Opening with `O_NOFOLLOW` accessed the link itself, not the target.

Test 5 – Unlinking Symlink:

Removing a symbolic link did not delete the target file.

Test 6 – Chained Links:

Multiple chained links resolved correctly to the final target.

Conclusion

In Task 5.1, the xv6 file system was enhanced to support doubly-indirect block addressing, allowing files to exceed the original 268 KB size limit. A new doubly-indirect pointer was added to the inode structure, enabling multi-level block indexing. This modification significantly increased the maximum file size and improved xv6's scalability for larger data storage. In Task 5.2, symbolic link (soft link) support was implemented by introducing a new inode type `T_SYMLINK` and a system call `symlink(target, path)`. The `open()` system call was extended to recursively resolve symbolic links with a defined depth limit, while supporting the `O_NOFOLLOW` flag for direct link access. Comprehensive testing confirmed correct functionality for file linking, reading, writing, unlinking, circular link detection, and recursive link resolution.